Smart Contract Security Assessment

25 August 2025

Project Name:

Spire: DA Builder Audit Report

Prepared by:

HYDN

Executive Summary

Client Name: Spire: DA Builder

Language: Solidity

Timeline: Delivered 25th August 2025

Method: Static Automated Analysis + Manual Review

Contents

Executive Summary 2
Contents 3
Introduction 3
About HYDN 4
Methodology 4
Basic Coding Mistakes 5
Business Logic Risks 5
Complex Integration Risks 6
Code Maturity 6
Impact Rankings 6
Disclaimer 7
Summary of Findings 8
Executive Summary 8
Project Summary 8
Breakdown of Findings 8
Detailed Findings 9
1 Missing Event Dispatch on Variable Update [Info] 9
2 Missing Event on Charge Action [Info] 10
3 Extra Gas Cost by Variable Assignment: Misleading Event Variable [Info] 11

Introduction

About HYDN

HYDN is an industry leader in blockchain security and smart contract audits. HYDN offers industry leading Smart Contract Audits and Web3 Penetration Testing for EVM compatible blockchains. Alongside this, HYDN also offers Consulting, Concept, Business Modelling, Validation, and Technical Design for blockchain projects. HYDN was the winner of the EthernautDAO CTF Hacking Challenge 2022, hacking the smart contracts twice in under 5 minutes. HYDN's Lead Auditor also placed as the highest solo competitor in the Paradigm CTF 2023.

HYDN have previously worked with the likes of Metamask, SushiSwap, Bittrex Global, Sablier, Revert Finance, Telos Foundation, StakeDAO, Swapsicle, My Lovely Planet, and more.

HYDN worked closely with Spire to consider their unique business needs, objectives, and concerns. Our mission is to ensure the blockchain supports secure business operations for all and he has built the team at HYDN from the ground up to meet this very personal objective.

To keep up to date with our latest news and announcements, check out our website https://hydnsec.com/ or follow @hydnsecurity on Twitter.

Methodology

When tasked with conducting a security assessment, HYDN works through multiple phases of security auditing to ensure smart contracts are audited thoroughly. To begin the process automated tests are carried out, before HYDN then moves onto carrying out a detailed manual review of the smart contracts.

HYDN uses a variety of open-source tools and analyzers as and when they are required and alongside this, HYDN primarily focuses on the following classes of security and reliability issues:

Basic Coding Mistakes

One of the most common causes of critical vulnerabilities is basic coding mistakes. Countless projects have experienced hacks and exploits due to simple, surface level mistakes that could have been flagged and addressed by a simple code review. The HYDN automated audit process which includes model checkers, fuzzers, and theorem provers analyses the smart contract for many of these basic coding mistakes. Once the automated audit has taken place, HYDN then performs a manual review of the code to gain familiarity with the contracts.

Business Logic Risks

HYDN reviews the platform or projects design documents before analysing the code to ensure that the team has a deep understanding of the business logic and goals. Following this, HYDN reviews the smart contracts to ensure that the contract logic is in line with the expected functionality.

HYDN also analyses the code for inconsistencies, flaws, vulnerabilities, or non-technical business risks which could impact business logic such as tokenomics errors, arbitrage opportunities, and share pricing.

Complex Integration Risks

Several high-profile exploits have been the result of not any bug within the contract itself, but rather an unintended consequence of its interaction with the broader DeFi ecosystem.
We perform a meticulous review of all of the contract's possible external interactions, and summarise the associated risks; for example: flash loan attacks, oracle price manipulation, MEV/sandwich attacks, etc.

Code Maturity

HYDN reviews the smart contracts for potential improvements in the codebase. These improvements ensure that the code follows industry best practices and guidelines, or code quality standards. Alongside this, HYDN makes suggestions for code optimization items such as gas optimization, upgradeability weaknesses, centralization risks, and more.

Impact Rankings

Once HYDN has completed the security assessment, each issue is assigned an impact rating. This impact rating is based upon both the severity and likelihood of the issue occurring. Each security assessment is different and the business needs and goals, such as project timelines, and Spire threat modelling. are taken into account when the impact rankings are created.

HYDN assigns the following impact rankings: Critical, High, Medium, Low (listed by severity).

Disclaimer

This HYDN security assessment does not provide any warranties on finding all possible issues within the scope and the evaluation results do not guarantee the absence of any issues. **HYDN cannot make any guarantees on any further code which is added or altered after the security assessment has taken place**. As a single security assessment can never be considered fully comprehensive, HYDN always recommends multiple independent assessments paired with a bug bounty program. This security assessment report should not be considered as financial or investment advice.

Summary of Findings

Executive Summary

The security audit of the Spire DA Builder Contract was carried out between August 11th - August 25th 2025 by HYDN. The testing consisted of both manual and automated testing and to kick off the audit HYDN first gathered a deep understanding of the business goals and intended actions of the contract and Spire team.

During the audit, HYDN found the code to be of a high quality standard and found only Informational severity issues and gas optimization recommendations.

Project Summary

Delivery Date	August 25th 2025
Language	Solidity

Repository	https://github.com/spire-labs/da-builder-contract
Initial Commits	0389c03eb742b5fdb6b3a9f1f80389acfd589b3c
Scope	*https://github.com/spire-labs/da-builder-contract*
Latest Commits	TBC

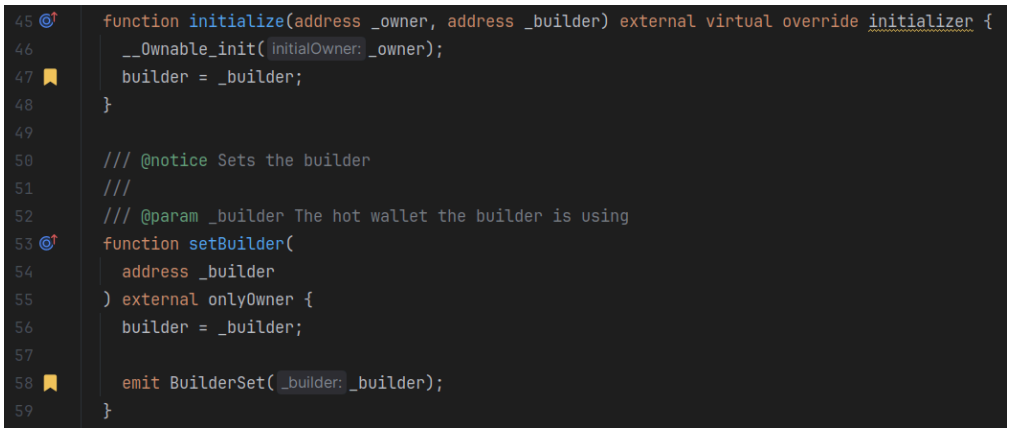
Breakdown of Findings

ID	Title	Severity	Remediation
1	Missing Event Dispatch on Variable Update	**Info**	Open
2	Missing Event on Charge Action	**Info**	Open
3	Extra Gas Cost by Variable Assignment: Misleading Event Variable	**Info**	Open

Detailed Findings

1 Missing Event Dispatch on Variable Update [Info]

Description	On the contract GasTank.sol, the function **initialize** updates the address value of the address without emitting an event. When the value is updated on the other function, **setBuilder**, an event is emitted with the final address.

	
	`function initialize(address _owner, address _builder) external virtual override initializer {`
	`  __Ownable_init(initialOwner: _owner);`
	`  builder = _builder;`
	`}`
	
	`/// @notice Sets the builder`
	`///`
	`/// @param _builder The hot wallet the builder is using`
	`function setBuilder(`
	`  address _builder`
	`) external onlyOwner {`
	`  builder = _builder;`
	
	`  emit BuilderSet(_builder: _builder);`
	`}`

Impact	If any off-chain system keeps track of the value by reading the events, there will be a period of time until action is taken to set a different value.
Recommendation	For consistency, and to be able to keep track of the history through the events, HYDN recommends adding the event also on the **initialize** function, or calling **setBuilder** from inside initialize.
Remediation	TBC

2 Missing Event on Charge Action [Info]

Description	An important event that changes state must emit an event for accounting, watching, and simple indexing. Performance motivation may balance in

	opposite directions, such as the **call()** function, but **charge()** should not be called very often, as it is not time or performance-critical, it could emit an event with no drawdown.
	
	`function _charge(Account calldata _account, address _recipient) internal {`
	`  balances[_account.operator] -= _account.charge;`
	`  (bool _success,) = _recipient.call{value: _account.charge}( bytes: '');`
	`  if (!_success) {`
	`  revert FailedLowLevelCall();`
	`  }`
	`}`

Impact	
Recommendation	HYDN recommends to emit an event when a user is charged.
Remediation	TBC

3 Extra Gas Cost by Variable Assignment: Misleading Event Variable [Info]

Description	In the contract GasTank.sol, the function **_deposit**, on Line 152, the **variable _balance** is initialized with the value of the balance of the

operator. This variable is only used a single time, so there is no benefit in assigning this value to a new variable.

In the same way, **_newBalance** is also created and assigned the final balance of the user after the increment of **msg.value**.

While this is done so that the final balance can be logged on the event, we argue that since the event is called "***AccountDeposited***" it would make more sense to log **msg.value** instead of the final amount.

```
function _deposit(
  address _operator
) internal {
  if (withdrawalStartedAt[_operator] != 0) revert AccountClosing();

  uint256 _balance = balances[_operator];
  uint256 _newBalance = _balance + msg.value;

  balances[_operator] = _newBalance;

  emit AccountDeposited( _operator: _operator,  _newBalance: _newBalance);
}
```

Impact	The impact of this finding is the extra gas cost for the two variable assignments - specifically, two **MSTORE** operations.
Recommendation	If the off-chain system can still keep track of the virtual balance of the user by checking the deltas, instead of the absolutes, we recommend removing both variable assignments and simply logging **msg.value**.

	If logging that value is a must, we recommend at least removing the creation of the variable **_balance**. A second approach would be to fix the semantics with a more appropriate event like **AccountBalanceUpdated** (*address owner, uint256 newBalance*)
Remediation	TBC